In [1]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
```

In [2]:
```
print("Loading MNIST dataset...")

mnist=fetch_openml("mnist_784",version=1,as_frame=False)

X=mnist.data.astype("float32")
y=mnist.target.astype(int)

print("Dataset shape:",X.shape)
```

```
Loading MNIST dataset...
Dataset shape: (70000, 784)
```

In [3]:
```
X=X/255.0
```

In [4]:
```
X=X[:3000]
y=y[:3000]
```

In [5]:
```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

In [8]:
```
scalar=StandardScaler()
X_train=scalar.fit_transform(X_train)
X_test=scalar.transform(X_test)
```

In [9]:
```
linear_svm=SVC(kernel="linear")
linear_svm.fit(X_train,y_train)
y_pred_linear=linear_svm.predict(X_test)
print("Linear SVM")
linear_accuracy=accuracy_score(y_test,y_pred_linear)
print("Linear Accuracy: ",linear_accuracy)
```

```
Linear SVM
Linear Accuracy:  0.9183333333333333
```

In [14]:
```
poly_svm=SVC(kernel="poly",degree=3)
poly_svm.fit(X_train,y_train)
y_pred_poly=poly_svm.predict(X_test)
print("Polynomial SVM")
poly_accuracy=accuracy_score(y_test,y_pred_poly)
print("Polynomial Accuracy: ",poly_accuracy)
```

```
Polynomial SVM
Polynomial Accuracy:  0.8066666666666666
```

In [15]:
```
rbf_svm=SVC(kernel="rbf")
rbf_svm.fit(X_train,y_train)
y_pred_rbf=rbf_svm.predict(X_test)
```

```
print("RBF SVM")
rbf_accuracy=accuracy_score(y_test,y_pred_rbf)
print("RBF Accuracy: ",rbf_accuracy)
```

```
RBF SVM
RBF Accuracy:  0.9183333333333333
```

In [16]:

```
result=pd.DataFrame({
    "Model":["Linear","Polynomial","RBF"],
    "Accuarcy":[
        accuracy_score(y_test,y_pred_linear),
        accuracy_score(y_test,y_pred_poly),
        accuracy_score(y_test,y_pred_rbf)]
})
print(result)
```

```
        Model  Accuarcy
0      Linear  0.918333
1  Polynomial  0.806667
2         RBF  0.918333
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: